

Q-COMMERCE ANALYTICS • PORTFOLIO PROJECT

# Beyond Fill Rate

*94% fill rate. One in three orders broken.*

*Both numbers are correct.*

A warehouse, an ambiguous question, and three mistakes worth keeping.

Akanksha Swarnim • Python • dbt • DuckDB • SQL • Synthetic data

# The question, as it arrived

*"Why is retention falling when our fill rate has stayed above 95%?"*

— Operations

## They were right.

Fill rate never dropped below 95%. They were reading a true number that could not answer their question.

- Not a data bug — nobody logged anything wrong.
- Not an ops failure — nobody was doing their job badly.
- A grain problem. The metric measures items. The customer buys a basket.

The most dangerous number is not the wrong one. It is the true one that answers a different question.

1

# Why I chose this problem

Disclosure first — it shapes everything that follows

## 2014-15 • PepperTap

I ran customer experience at an on-demand grocery startup — launch to 20,000+ orders a day across 20 cities.

*“Every stock-out in this project is one I have personally apologised for.”*

I knew what a stock-out felt like on the phones. I could not measure what it cost.

## ...and that is also the bias

I chose stock-outs because I know that problem in my hands — not because I proved it was the most valuable question available.

Someone with rider-ops experience would have built the delivery-lateness version of this project.

They might have been more right.

Domain experience is what let me pick a real problem. Stating where it biases me is what keeps it honest — see the limitations slide.

## 2 Why stock-outs are structural, not a bug

The 15-minute promise forces everything else

|              | Supermarket      | Next-day e-grocery | Quick commerce      |
|--------------|------------------|--------------------|---------------------|
| Promise      | you drive there  | tomorrow, 2h slot  | ~15 minutes         |
| Assortment   | 30,000+ SKUs     | 20,000+            | 2,000–4,000         |
| Fulfilment   | customer picks   | regional warehouse | darkstore, ~2 km    |
| Basket       | large, planned   | large, planned     | small, urgent       |
| Substitution | customer decides | pre-approved       | picker decides, now |

A darkstore is small ON PURPOSE. 15 minutes means expensive urban space, means ~2,500 SKUs where a supermarket holds 30,000.

So the assortment is thin by design.

**Stock-outs are not a failure of the model. They are the model.**

You cannot inventory your way out without buying space that destroys the unit economics — the constraint the promise created.

3

## Why this needs a warehouse, not a query

Answering it requires eight systems that disagree about everything

| Source system              | Grain                | Clock | Latency               | Owner           |
|----------------------------|----------------------|-------|-----------------------|-----------------|
| Order service (events)     | one row per EVENT    | UTC   | real-time + retries   | Commerce        |
| Order service (items)      | order × product      | —     | real-time             | Commerce        |
| Warehouse ops (fulfilment) | one row per order    | UTC   | up to 6h late         | Warehouse Ops   |
| App analytics (sessions)   | one row per session  | UTC   | real-time             | Product         |
| Catalogue                  | product × version    | —     | daily (SCD2)          | Catalogue       |
| Ops master data            | store × version      | —     | ad hoc (SCD2)         | Operations      |
| CRM                        | one row per customer | —     | daily                 | Marketing       |
| Experiment platform        | one row per session  | UTC   | assignment ≠ exposure | Experimentation |

Three grains. Two latencies. One timezone mismatch. Two slowly-changing dimensions. The business speaks GST; every system logs UTC. This is not a query — it is a data model.

*The warehouse exists to connect a missing item at 19:04 on a Tuesday to a customer who quietly stopped ordering in March. No single system can do that.*

4

# The workflow

Eight stages, ~90 seconds, no cloud account — `./run_all.sh`

## 1 Generate

8 synthetic source systems + 8 defects

## 2 Land

raw schema — defects intact

## 3 Model + test

staging → marts  
45 dbt tests

## 4 Analyse

BLIND to the answer key

## 5 Score

validate.py vs ground truth

## 6 Render docs

prose generated from results

## 7 Dashboard

+ simulator, from results

## 8 Verify

docs match results, or fail

Stage 4 and stage 5 are separate FILES on purpose. analysis.py raises if it can reach the generative parameters; validate.py is the only file allowed to open them, and it runs after results are on disk.

*Estimate first, grade second. Collapsing those two is how a synthetic demo quietly becomes a tautology.*

5

# The constraints I worked under

Each one shaped a decision, not just an apology



## No real data

No operator will hand you their order log. So: generate it — and declare every injected parameter in one file BEFORE analysing.



## No cloud account

BigQuery costs money. So: DuckDB, held to the dialect overlap (CTEs, window functions, QUALIFY). What would differ is documented, not faked.



## No satisfaction label

Nothing records whether a substitution was OK. So: three proxies, and report their disagreement rather than pick one.



## No cost data

Every recommendation implies a spend. So: state assumptions explicitly and show what would change the answer — never invent an ROI.



## One pair of eyes

No peer review. Every test written by the person whose code it tests. So: document the mistakes where they happened.

*A constraint you name is a design decision. A constraint you hide is a bug waiting to be found by someone else.*

6

## No analysis until the data is trustworthy

Eight defects planted on purpose — each with a dbt test that catches it

### Duplicate events

*client retry, no idempotency key*

1,776 dupes → deduped with QUALIFY

### Late-arriving data

*fulfilment trails by up to 6h*

7.9% of rows · worst lag 359 min

### UTC vs GST

*business reports GST, systems log UTC*

14.2% of orders land on the wrong day

### SCD2 price/catchment

*naive join restates history*

half-open range join + overlap & gap tests

### QA bot accounts

*flag exists, never populated*

found behaviourally · precision 1.00

### Missing category (3%)

*catalogue join failure*

→ 'Unmapped', never filtered

### Cancelled orders

*in Finance, out of behaviour*

definition documented, not buried

### Grain trap

*item vs order fill rate*

encoded as a dbt INVARIANT

Most projects fix defects. This one MEASURES them — 45 dbt tests, all green. The invariant test asserts order fill can never exceed item fill, so the pipeline defends the finding rather than the README asserting it.



7

## Finding 1 — the grain trap

The same data, measured at two grains

94.09%

ITEM fill rate  
what the shift report shows

66.85%

ORDER fill rate  
what the customer experiences

Same 135,996 orders. Same 1,054,890 items. A gap of 27.2 points — and both are correct.

$$P(\text{clean order}) = (1 - \text{item miss rate})^{\text{basket size}}$$

At a 5.9% miss rate and a 7.8-item basket, roughly a third of orders are broken — by arithmetic.

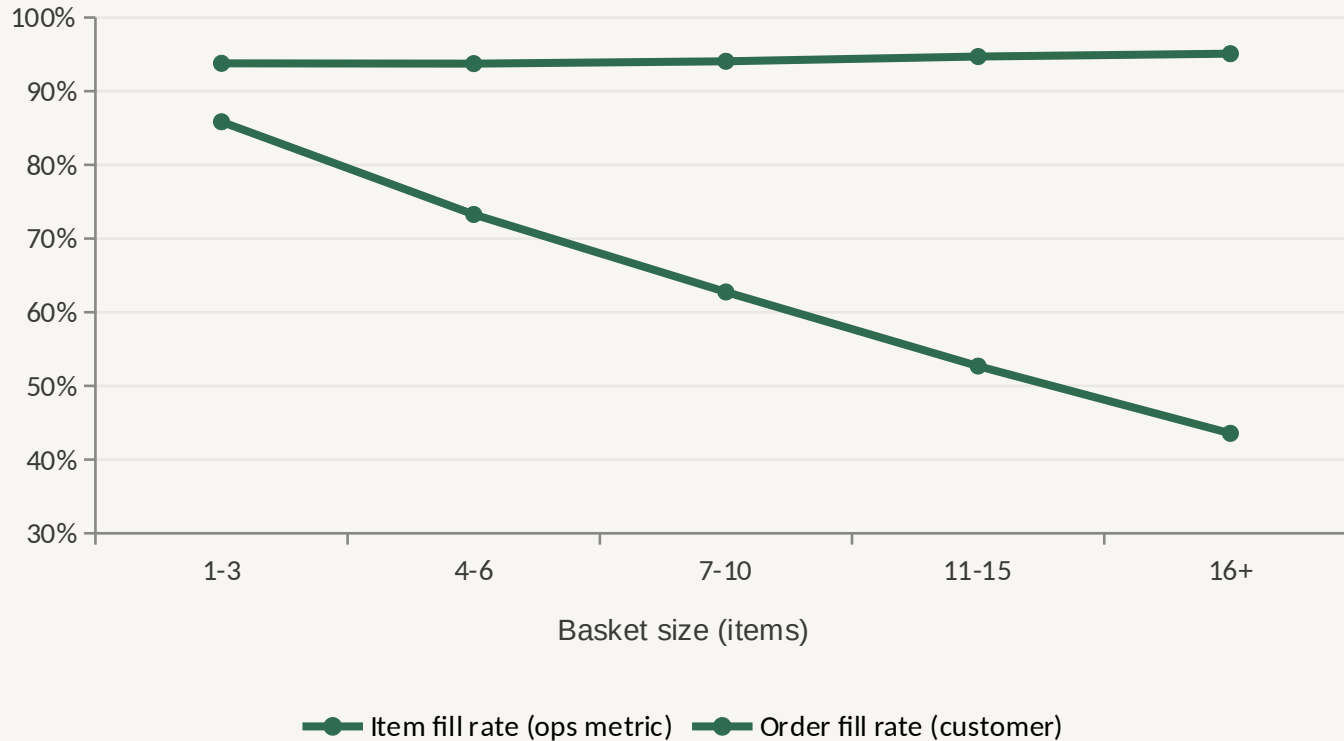
An order is clean only if EVERY item is. Nobody is doing their job badly — the metric measures items, and nobody buys an item. They buy a basket.

*This finding only exists because the warehouse holds both grains. Model at order grain to 'save space' and it is unrecoverable.*

8

## Finding 1b — the inversion is the decision

Read the two lines against each other



**Same period.**  
**Same data.**  
**Opposite conclusions.**

Item fill rate says large baskets are served **SLIGHTLY BETTER** (93.8% → 95.1%).

Order fill rate says **TWICE AS BADLY** (85.9% → 43.6%).

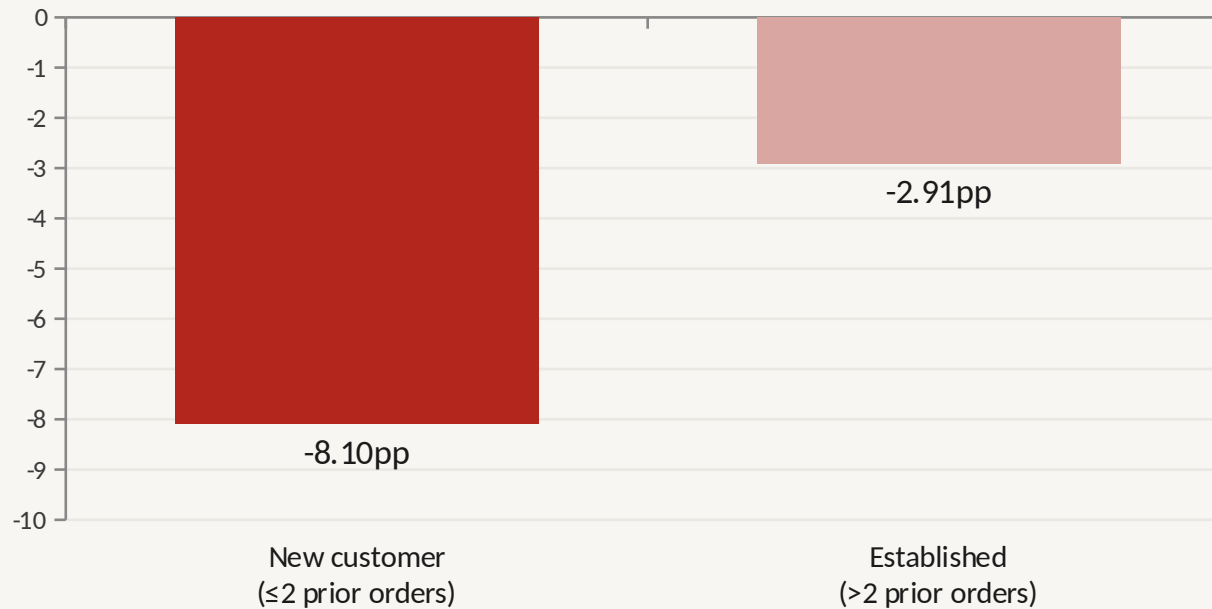
Large baskets are the Weekly Stock-Up segment — the highest-value customers on the platform.

The metric Operations optimises conceals that its best customers get its worst experience — and conceals it **MORE** the more they spend.

9

## Finding 2 — what a broken order costs

Effect on 30-day repeat rate, with 95% confidence intervals



≈ 3×

more damage on a first basket

A stock-out on order #1 is evidence the service does not work.

On order #30 it is a Tuesday — they have 29 orders' worth of prior saying you are fine.

**New customers are 68% of the orders we can measure.**

The pooled figure (-7.04 pp) averages two populations that need OPPOSITE decisions. It should not be on a dashboard.

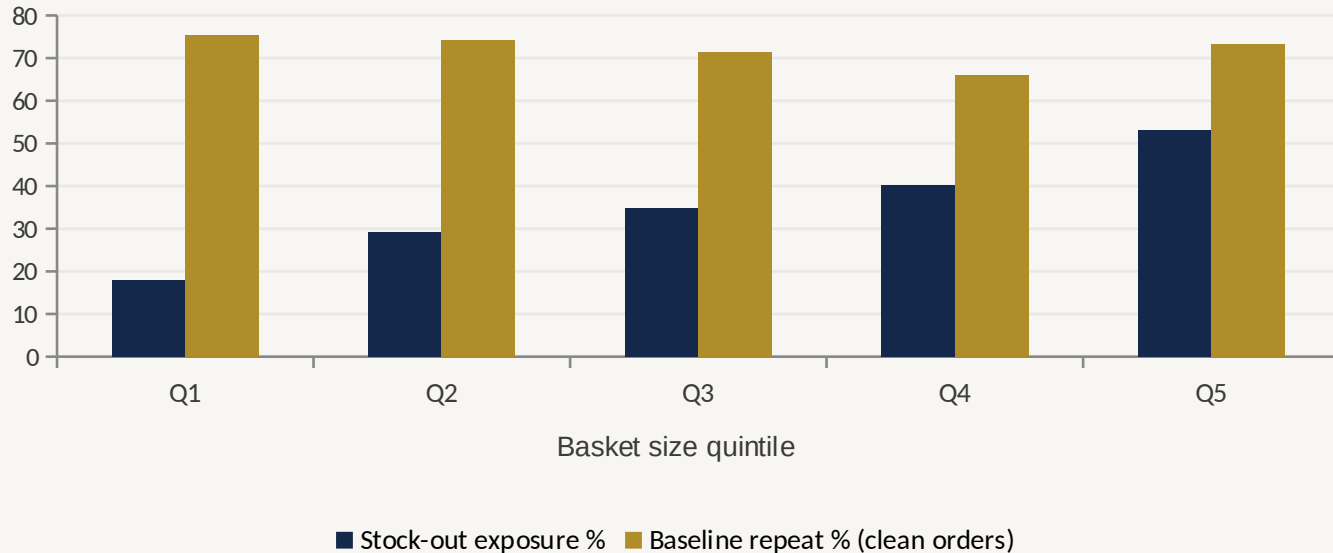
Observational, not causal — stock-outs were not randomised. These are associations under a stated adjustment set.

# The hypothesis I was wrong about

Published as a negative result rather than deleted

## MY HYPOTHESIS

Heavy users order more → more exposed to stock-outs → and they retain better anyway. So the naive estimate is biased toward zero, hiding real damage.



**Exposure TRIPLES.**

**Retention is FLAT.**

Basket size is a strong EXPOSURE confounder and a weak OUTCOME confounder. Adjusting for it barely moves anything:

|                   |          |
|-------------------|----------|
| naive             | -7.04 pp |
| basket-stratified | -6.60 pp |

**Hypothesis: REJECTED.**

An analysis that only shows the adjustments that WORKED is not an analysis. It is an argument. This also pays off later — it is why the experiment's selection bug does not manufacture a fake win.

11

# The ambiguous question

"Are we substituting well?" — reasonable, and unanswerable

**Nothing in eight source systems records whether the customer was OK with what arrived.**

No satisfaction field. No thumbs-up. No rating. The picker's swap is logged. The customer's reaction is not.

So it cannot be answered. It can only be REPLACED — and the replacement is the judgement.

A

Acceptance as consent

3.49 pp

Blind spot: a customer at 11pm with no alternative accepts anything. Acceptance ≠ endorsement.

B

Retention as forgiveness

4.07 pp

Blind spot: whether a substitute was OFFERED is a picker decision, not a coin flip.

C

Swap distance

+1.10 to +5.70 pp

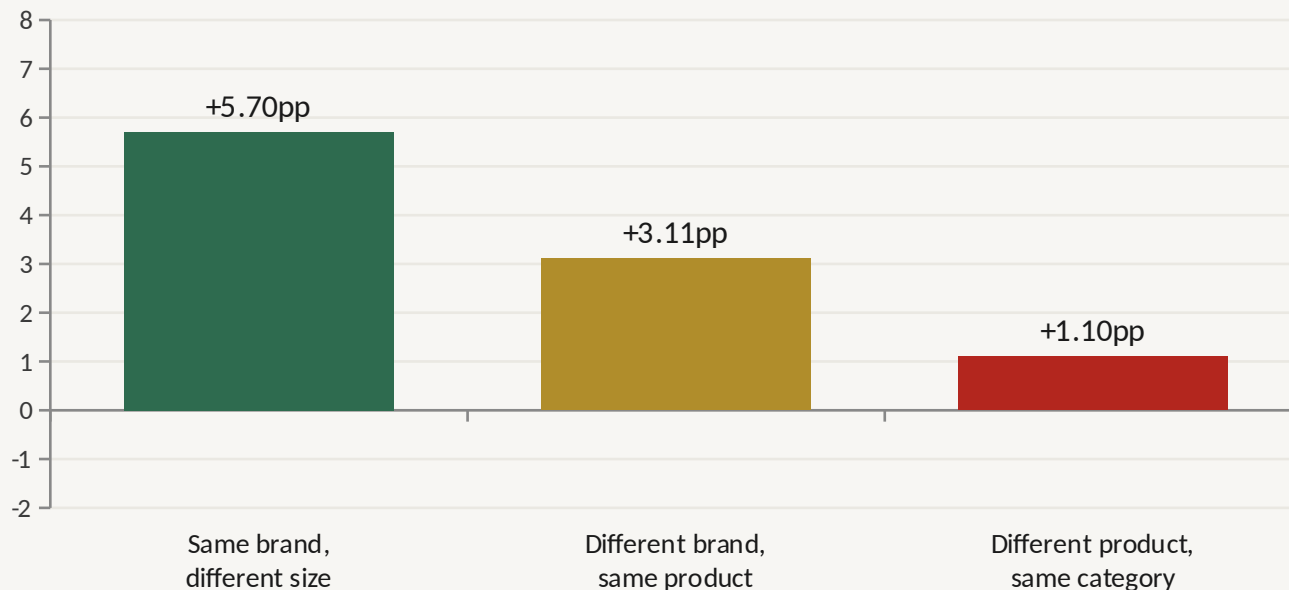
Blind spot: this encodes a belief about groceries, not a measurement of customers.

They disagree by 4.6 pp. Reporting any one as "the impact of substitutions" would be choosing a number and calling it a measurement.

12

## What survives the ambiguity

The magnitude is unrecoverable. The ranking is not.



### Order stable across BOTH control groups

Tested against two controls with OPPOSITE selection problems — one holds the picker's decision fixed, the other holds the customer's choice fixed.

Agreement means something precisely because they do not share a confounder.

Injected truth:  $0.82 > 0.47 > 0.11$

Recovered order: identical.

The business decision was never a magnitude decision. Nobody acts on "substitutions recover 4.1%" — they act on "which swap do I make?" That is a ranking question, and the ranking holds.

Milk → laban: 1.10 pp, CI [-1.44, 3.63] — spans zero. Statistically indistinguishable from offering nothing. A quarter of substitution offers are ritual. That is not a null result; it is an instruction.

## 13 Finding 3 — the experiment that looked fine

Pre-approved substitutions: assignment at session start, exposure at checkout

| Check                      | Result               | Reading                                     |
|----------------------------|----------------------|---------------------------------------------|
| SRM at assignment          | $p = 0.77$           | randomisation performed correctly           |
| SRM at checkout            | $p = 3e-153$         | randomisation did NOT survive               |
| Basket at checkout         | 7.92 vs 7.27         | the treatment selected who reached checkout |
| Retention among converters | -0.39 pp, $p = 0.30$ | "no effect — harmless, ship it"             |
| CONVERSION                 | -8.34 pp             | 19% of treatment conversions DESTROYED      |
| ITT (everyone assigned)    | -5.88 pp             | net harm                                    |

### It is not a fake win. It is a fake HARMLESS.

Retention-among-converters is conditioned on converting — the exact thing the treatment broke. The lost users are not in the numerator, the denominator, or the table.

A fake win gets scrutinised — someone always asks why the number is so good. NOBODY RE-EXAMINES A NULL. One chi-square test stood between "no effect" and "this is destroying the funnel". Decision: DO NOT SHIP.

14

# Three mistakes I kept in the code

The correction is the part worth reading

1

## My generator was wrong

Cancelled orders emitted only 'order\_cancelled', never 'order\_placed' — so they vanished from the fact table and the cancellation rate read as ZERO while the money moved.

*Caught because fct\_orders returned fewer rows than the source.*

2

## My bot detector had precision 0.041

Flagged 1,144 accounts to catch 47. My "conservative" threshold was an assumption about UNITS, not a fact about the data — median 2, MAD 1, so  $k=8$  meant ~13 orders.

*Re-measured: classes genuinely OVERLAP (human max 31, bot min 29). Recall is capped by the world, not by tuning. Landed at precision 1.00, recall 0.98.*

3

## I built a test that could not fail

Wrote docs, regenerated data, never re-read my prose. So I built a checker that greps each number and confirms it exists in results.json. It passed on stale docs TWICE.

*That is the item-fill-rate error one layer up, in my own tooling: technically correct, cheap, answering a different question. Docs are now GENERATED — drift is impossible.*

*I trust evidence more than my own assumptions — including the assumption that my test was working.*



# 15 What data science did, and what it could not

Neither half works alone

## DS What data science did

- Quantified a gap everyone felt and nobody measured — 27 points.
- Separated two populations the average was hiding (3× damage on first baskets).
- Proved the confound I feared did NOT bite — and said so.
- Caught an experiment that read as harmless and was destroying 19% of conversions.
- Showed the substitution ranking is robust while the magnitude is not.

## H What the human angle did

- Chose the question. No model tells you fill rate is the wrong metric.
- Knew stock-outs are structural, not fixable — so "improve fill rate" is a wish, not advice.
- Knew a missing formula at 11pm ≠ a missing bag of crisps. That flag is a judgement, and it is labelled as one.
- Knew the picker has ten seconds and three options — which made swap TYPE the unit of analysis.
- Named the bias: I picked stock-outs because I know them, not because I proved they matter most.

The statistics found the gap. The three years on the phones is what made me look at the grain in the first place — and what told me which judgement calls needed labelling as judgement.

16

# What I would actually recommend

Not "improve fill rate" — that is a wish, not advice

## 1 Change the headline metric

Report ORDER fill rate. Keep item fill for shift management — it is the right ops metric. Stop letting it answer customer questions.

**Costs nothing. Do it this week.**

## 2 Protect first baskets

Weight assortment depth and picker priority toward new-customer orders, where damage is ~3× larger.

**Costs real money — see break-even.**

## 3 Stop same-category substitutes

Milk → laban recovers nothing measurable. A quarter of substitution offers are ritual: picker time spent on nothing.

**Cost is NEGATIVE — frees capacity.**

## 4 Do not ship pre-approval

Destroys 19% of conversions and buys no retention. The idea is sound; the checkout interstitial kills it. Move it out of the purchase path.

**Saves a bad quarter.**

## 5 Ask the customer

One question after a substituted delivery: "was this OK?" Replaces three disagreeing proxies with an actual label.

**Should have existed before the dashboard.**

We have no cost data. So the memo states its assumptions and shows what would change the answer — rather than inventing an ROI and calling it a finding.

# What this does NOT establish

Placed here, not in an appendix

## The data is synthetic

The 94% / 67% gap is a property of a model I wrote — not a measurement of any operator. The MECHANISM transfers. The MAGNITUDES do not. Please do not quote them.

## I wrote both the generator and the analysis

Failing validation proves the code is wrong. PASSING DOES NOT PROVE IT RIGHT. It proves it is not-yet-known-to-be-broken on data whose DGP is a documented Python file.

## No rider data — the biggest gap

Lateness and stock-outs both spike at peak. Some of what I attribute to stock-outs probably belongs to lateness. I cannot separate them. These estimates are plausibly UPWARD-BIASED.

## The causal claims are observational

Stock-outs were not randomised. Associations under a stated adjustment set — not experimental effects. That is why the experiment layer exists.

## One author, one reviewer

Every dbt test written by the person whose code it tests. The interesting question is what is still in here that I have not noticed.

If this changes how someone thinks about the gap between an ops metric and a customer metric, it worked. If someone quotes 66.85% in a meeting, it did harm — and that is partly my fault for making it quotable.

# Five learnings

1

## **A true number can still be the wrong number.**

Item fill rate was never inaccurate. It answered a question nobody asked.

2

## **The grain IS the analysis.**

The entire finding lives in the gap between two grains. Model at the coarser one and it is unrecoverable — forever.

3

## **Report the adjustments that failed.**

The confound I feared did not bite. Saying so is what makes the ones that did bite believable.

4

## **Nobody re-examines a null.**

A fake win gets scrutinised. A fake harmless ships. Guardrails are not paperwork.

5

## **Ambiguity is not answered — it is decomposed.**

"Was it good?" has no answer. "Which swap is better?" does. Knowing which one you can afford to ask is the job.

# Beyond Fill Rate

The whole thing runs end to end in ~90 seconds.

8

source systems

45

dbt tests, all green

8

planted defects

0

hand-typed numbers

Python · dbt · DuckDB · SQL · every figure rendered from one results file

[github.com/AkankshaSwarnim/qcommerce-analytics-warehouse](https://github.com/AkankshaSwarnim/qcommerce-analytics-warehouse)

*If you have worked in quick commerce and think I have the mechanism wrong — particularly on rider data — I would like to hear it.*

Akanksha Swarnim · Data Scientist, Dubai · Synthetic data